

```
import os
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import sklearn.preprocessing
from sklearn.metrics import r2_score

from keras.layers import Dense,Dropout,SimpleRNN,LSTM
from keras.models import Sequential

#check all the files in the input dataset
#print(os.listdir("../input/"))
```

2. Data loading and data exploration

• Load the data file

```
#choosing DOM_hourly.csv data for analysis
fpath='D://Users//user//Downloads//DOM_hourly.csv//DOM_hourly.csv'

df=pd.read_csv(fpath)
df.head()
```

	Datetime	DOM_MW
0	2005-12-31 01:00:00	9389.0
1	2005-12-31 02:00:00	9070.0
2	2005-12-31 03:00:00	9001.0
3	2005-12-31 04:00:00	9042.0
4	2005-12-31 05:00:00	9132.0

• Change the index of rows in the dataframe from 0,1,2... to datetime (2005-12-31 01:00:00,...)

Why should we change the index of rows?

Because we are dealing with time series data and we will need the datetime data to recognize a particular record.

```
#Let's use datetime(2012-10-01 12:00:00,...) as index instead of numbers(0,1,...)
#This will be helpful for further data analysis as we are dealing with time series data
df = pd.read_csv(fpath, index_col='Datetime', parse_dates=['Datetime'])
df.head()
```

	Datetime	DOM_MW
2005-12-31 01:00:00		9389.0
2005-12-31 02:00:00		9070.0
2005-12-31 03:00:00		9001.0
2005-12-31 04:00:00		9042.0
2005-12-31 05:00:00		9132.0

• Check if there are missing values in the data loaded

```
#checking missing data
df.isna().sum()
```

```
DOM_MW    0
dtype: int64
```

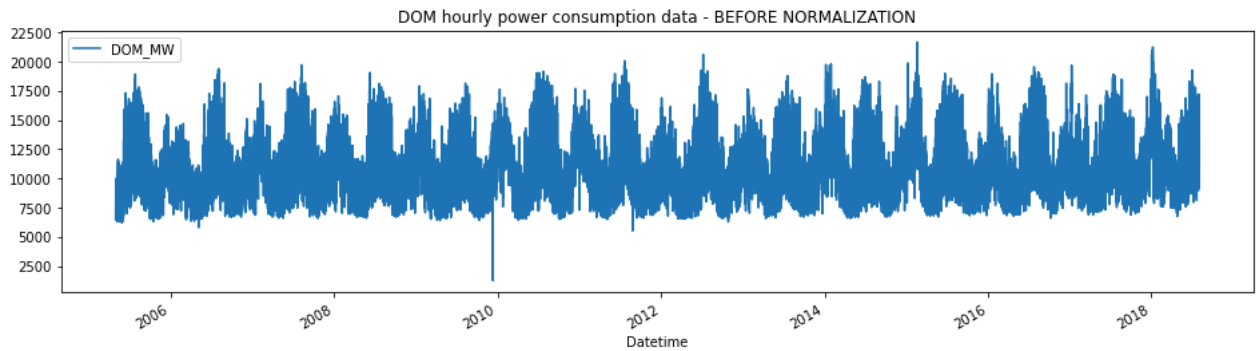
Since there is no missing data in the data loaded we will not be dropping the missing value records or will not be imputing the data. We will proceed with the further data analysis.

• Data visualization

```
df.plot(figsize=(16,4),legend=True)

plt.title('DOM hourly power consumption data - BEFORE NORMALIZATION')
```

```
plt.show()
```



- **Normalize data**
- Before proceeding with further data analysis we must ensure that the data is normalized.
- For this we will be using [sklearn MinMaxScaler](#)

```
def normalize_data(df):
    scaler = sklearn.preprocessing.MinMaxScaler()
    df['DOM_MW']=scaler.fit_transform(df['DOM_MW'].values.reshape(-1,1))
    return df
```

```
df_norm = normalize_data(df)
df_norm.shape
```

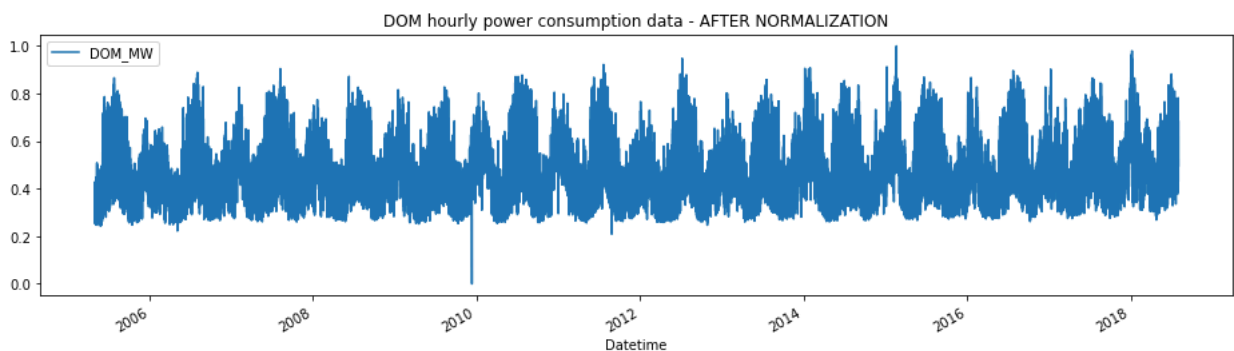
```
(116189, 1)
```

- **Visualize data after normalization**
- After normalization the range of power consumption values changes which we can observe on the **y-axis** of the graph. In the earlier graph that was displayed it was in the range **0 - 22500**
- Now after normalization we can observe that the data range on **y-axis** is **0.0 - 1.0**

```
df_norm.plot(figsize=(16,4),legend=True)

plt.title('DOM hourly power consumption data - AFTER NORMALIZATION')

plt.show()
```



```
df_norm.shape
```

```
(116189, 1)
```

3. Prepare data for training the RNN models

```
def load_data(stock, seq_len):
    X_train = []
    y_train = []
    for i in range(seq_len, len(stock)):
        X_train.append(stock.iloc[i-seq_len : i, 0])
        y_train.append(stock.iloc[i, 0])

    #1 last 6189 days are going to be used in test
    X_test = X_train[110000:]
    y_test = y_train[110000:]
```

```
#2 first 110000 days are going to be used in training
X_train = X_train[:110000]
y_train = y_train[:110000]

#3 convert to numpy array
X_train = np.array(X_train)
y_train = np.array(y_train)
print(X_train.shape)
X_test = np.array(X_test)
y_test = np.array(y_test)

#4 reshape data to input into RNN models
X_train = np.reshape(X_train, (110000, seq_len, 1))
print(X_train.shape)

X_test = np.reshape(X_test, (X_test.shape[0], seq_len, 1))

return [X_train, y_train, X_test, y_test]
```

To get an understanding on how sequence length is useful in training RNN models refer to the following links:

- <https://stackoverflow.com/questions/49573242/what-is-sequence-length-in-lstm>
- <https://stats.stackexchange.com/questions/158834/what-is-a-feasible-sequence-length-for-an-rnn-to-model>

```
#create train, test data
seq_len = 20 #choose sequence length

X_train, y_train, X_test, y_test = load_data(df, seq_len)

print('X_train.shape = ', X_train.shape)
print('y_train.shape = ', y_train.shape)
print('X_test.shape = ', X_test.shape)
print('y_test.shape = ', y_test.shape)

(110000, 20)
(110000, 20, 1)
X_train.shape = (110000, 20, 1)
y_train.shape = (110000,)
X_test.shape = (6169, 20, 1)
y_test.shape = (6169,)
```

4. Build a SIMPLE RNN model

```
rnn_model = Sequential()

rnn_model.add(SimpleRNN(40,activation="tanh",return_sequences=True, input_shape=(X_train.shape[1],1)))
rnn_model.add(Dropout(0.15))

rnn_model.add(SimpleRNN(40,activation="tanh",return_sequences=True))
rnn_model.add(Dropout(0.15))

rnn_model.add(SimpleRNN(40,activation="tanh",return_sequences=False))
rnn_model.add(Dropout(0.15))

rnn_model.add(Dense(1))

rnn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
simple_rnn (SimpleRNN)	(None, 20, 40)	1680
dropout (Dropout)	(None, 20, 40)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 40)	3240
dropout_1 (Dropout)	(None, 20, 40)	0
simple_rnn_2 (SimpleRNN)	(None, 40)	3240
dropout_2 (Dropout)	(None, 40)	0
dense (Dense)	(None, 1)	41

```
=====
Total params: 8,201
Trainable params: 8,201
Non-trainable params: 0
```

```
print(input_shape)
```

```
-----
NameError                                Traceback (most recent call last)
Input In [26], in <cell line: 1>()
----> 1 print(input_shape)

NameError: name 'input_shape' is not defined
```

```
rnn_model.compile(optimizer="adam",loss="MSE")
rnn_model.fit(X_train, y_train, epochs=10, batch_size=1000)
```

```
Epoch 1/10
110/110 [=====] - 16s 115ms/step - loss: 0.0586
Epoch 2/10
110/110 [=====] - 13s 118ms/step - loss: 0.0128
Epoch 3/10
110/110 [=====] - 13s 117ms/step - loss: 0.0077
Epoch 4/10
110/110 [=====] - 13s 117ms/step - loss: 0.0058
Epoch 5/10
110/110 [=====] - 13s 117ms/step - loss: 0.0044
Epoch 6/10
110/110 [=====] - 13s 117ms/step - loss: 0.0037
Epoch 7/10
110/110 [=====] - 13s 117ms/step - loss: 0.0031
Epoch 8/10
110/110 [=====] - 13s 115ms/step - loss: 0.0027
Epoch 9/10
110/110 [=====] - 13s 117ms/step - loss: 0.0024
Epoch 10/10
110/110 [=====] - 13s 117ms/step - loss: 0.0022
<keras.callbacks.History at 0x255b294efd0>
```

- Let's check r2 score for the values predicted by the above trained SIMPLE RNN model
- For more info on r2 score refer [this](#)

```
rnn_predictions = rnn_model.predict(X_test)
```

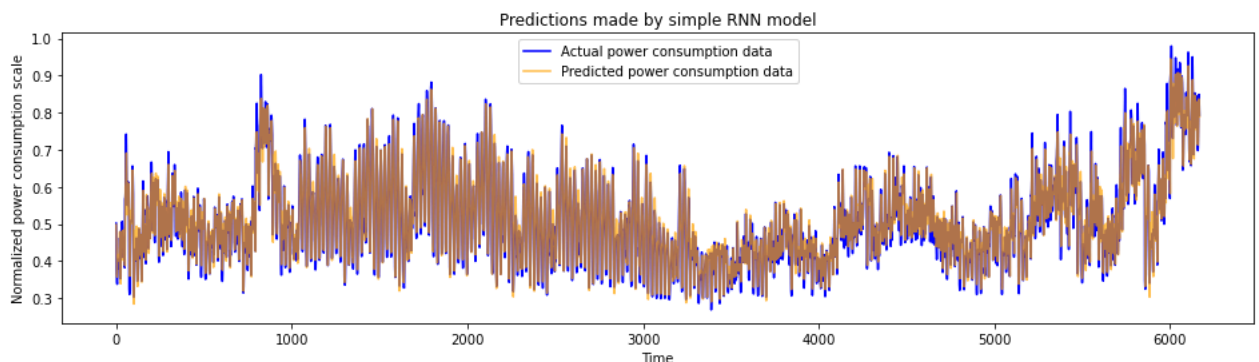
```
rnn_score = r2_score(y_test,rnn_predictions)
print("R2 Score of RNN model = ",rnn_score)
```

```
193/193 [=====] - 2s 7ms/step
R2 Score of RNN model = 0.9496818795006445
```

- Let's compare the actual values vs predicted values by plotting a graph
- We see that the predicted values are close to the actual values meaning the RNN model is performing well in predicting the sequence.

```
def plot_predictions(test, predicted, title):
    plt.figure(figsize=(16,4))
    plt.plot(test, color='blue',label='Actual power consumption data')
    plt.plot(predicted, alpha=0.7, color='orange',label='Predicted power consumption data')
    plt.title(title)
    plt.xlabel('Time')
    plt.ylabel('Normalized power consumption scale')
    plt.legend()
    plt.show()
```

```
plot_predictions(y_test, rnn_predictions, "Predictions made by simple RNN model")
```



5. Build an LSTM model

```
lstm_model = Sequential()

lstm_model.add(LSTM(40,activation="tanh",return_sequences=True, input_shape=(X_train.shape[1],1)))
lstm_model.add(Dropout(0.15))

lstm_model.add(LSTM(40,activation="tanh",return_sequences=True))
lstm_model.add(Dropout(0.15))

lstm_model.add(LSTM(40,activation="tanh",return_sequences=False))
lstm_model.add(Dropout(0.15))

lstm_model.add(Dense(1))

lstm_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 20, 40)	6720
dropout_3 (Dropout)	(None, 20, 40)	0
lstm_1 (LSTM)	(None, 20, 40)	12960
dropout_4 (Dropout)	(None, 20, 40)	0
lstm_2 (LSTM)	(None, 40)	12960
dropout_5 (Dropout)	(None, 40)	0
dense_1 (Dense)	(None, 1)	41
Total params: 32,681		
Trainable params: 32,681		
Non-trainable params: 0		

```
lstm_model.compile(optimizer="adam",loss="MSE")
lstm_model.fit(X_train, y_train, epochs=10, batch_size=1000)
```

```
Epoch 1/10
110/110 [=====] - 37s 260ms/step - loss: 0.0201
Epoch 2/10
110/110 [=====] - 29s 268ms/step - loss: 0.0118
Epoch 3/10
110/110 [=====] - 30s 274ms/step - loss: 0.0079
Epoch 4/10
110/110 [=====] - 29s 264ms/step - loss: 0.0047
Epoch 5/10
110/110 [=====] - 29s 262ms/step - loss: 0.0036
Epoch 6/10
110/110 [=====] - 29s 261ms/step - loss: 0.0028
Epoch 7/10
110/110 [=====] - 29s 260ms/step - loss: 0.0023
Epoch 8/10
110/110 [=====] - 29s 260ms/step - loss: 0.0021
Epoch 9/10
110/110 [=====] - 29s 261ms/step - loss: 0.0019
Epoch 10/10
110/110 [=====] - 29s 259ms/step - loss: 0.0018
<keras.callbacks.History at 0x255b2b94700>
```

- Let's check r2 score for the values predicted by the above trained LSTM model

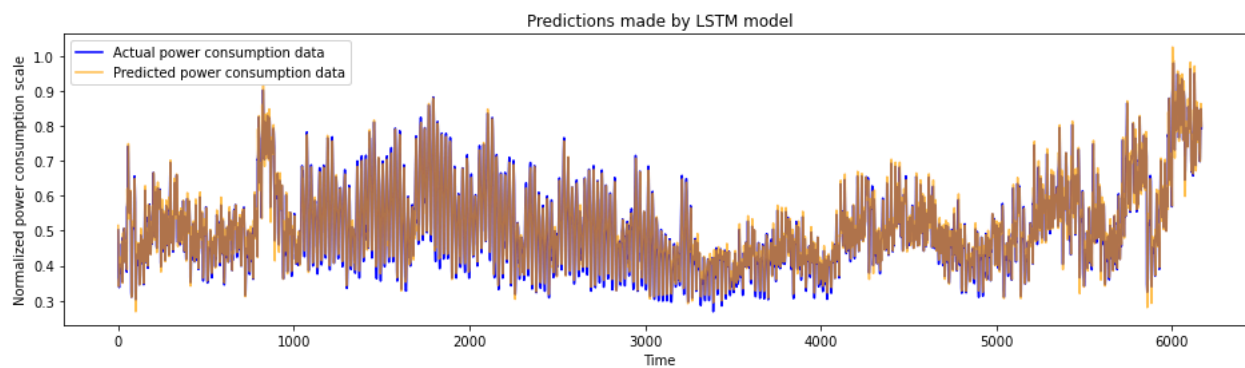
```
lstm_predictions = lstm_model.predict(X_test)

lstm_score = r2_score(y_test, lstm_predictions)
print("R^2 Score of LSTM model = ",lstm_score)
```

```
193/193 [=====] - 5s 14ms/step
R^2 Score of LSTM model = 0.9551185712490617
```

- Let's compare the actual values vs predicted values by plotting a graph

```
plot_predictions(y_test, lstm_predictions, "Predictions made by LSTM model")
```



6. Compare predictions made by simple RNN, LSTM model by plotting data in a single graph

```
plt.figure(figsize=(15,8))

plt.plot(y_test, c="orange", linewidth=3, label="Original values")
plt.plot(lstm_predictions, c="red", linewidth=3, label="LSTM predictions")
plt.plot(rnn_predictions, alpha=0.5, c="green", linewidth=3, label="RNN predictions")
plt.legend()
plt.title("Predictions vs actual data", fontsize=20)
plt.show()
```

